Errores frecuentes en Python y cómo arreglarlos

Tres pasos ante cualquier error: 1) busca el **número de línea** (`line 3`); 2) mira dónde apuntan las flechitas `^`; 3) lee la **última línea** del mensaje: dice el tipo de error. El problema está en esa línea o en la de antes.

Lo que dice Python	Qué significa	Ejemplo que lo provoca	Cómo se arregla
SyntaxError: unterminated string literal	Está mal escrito: un texto sin cerrar.	`print("¡Hola, mundo!)`	Cierra las comillas: `print("¡Hola, mundo!")`.
SyntaxError: expected ':'	Faltan los dos puntos.	`if nota >= 5`	Pon `:` al final de `if`, `elif`, `else`, `for`, `while` y `def`.
SyntaxError: invalid syntax. Maybe you meant '=='...	Has comparado con un solo `=`.	`if clave = "python":`	Para comparar, dos: `==`. Un solo `=` es para guardar.

IndentationError: expected an indented block after 'if' statement on line 2	Falta la sangría de lo que va dentro.	`edad = 15`
		`if edad >= 18:`
		`print("Mayor")`
			Mete 4 espacios la línea de dentro. Al pulsar `Enter` tras los dos puntos, el editor los pone solo.

IndentationError: unexpected indent	Sobra sangría: una línea empieza más adentro sin motivo.	`x = 5` `    print(x)`	Quita los espacios del principio de esa línea.
NameError: name 'nombr' is not defined. Did you mean: 'nombre'?	Python no conoce ese nombre.	`print(nombr)`	Escríbelo igual que cuando lo creaste (mira la sugerencia *Did you mean*). Si es una variable, tiene que existir antes de usarla.
TypeError: can only concatenate str (not "int") to str	Estás sumando texto y número.	`a = input("Número: ")` `print(a + 1)`	Convierte lo que da `input()`: `a = int(input("Número: "))`.
ValueError: invalid literal for int() with base 10: 'quince'	Se ha escrito una palabra donde iba un número.	El programa usa `int(input(…))` y el usuario escribe `quince`.	El programa está bien: vuelve a ejecutarlo y escribe `15`.

Sin mensaje de error, pero sale mal

Lo que pasa	Causa	Cómo se arregla
Con 4 y 6 sale `46` en vez de `10`.	No has convertido lo que da `input()`: ha pegado dos textos.	`int(input(…))` o `float(input(…))`.
La suma de un bucle sale con el último número, o sale varias veces.	`total = 0` está dentro del bucle, o el `print` tiene sangría.	`total = 0` antes del bucle y el `print` después, sin sangría.
La tabla del 6 llega a 6 x 9.	`range(10)` no incluye el 10.	`range(11)` o `range(0, 11)`: el último número no entra.
El programa no termina nunca.	Bucle infinito: la condición del `while` no deja de cumplirse.	En el editor online, recarga la página; en el editor de la prueba (`/pyodide`), pulsa el botón rojo Stop. Después revisa que dentro del bucle algo cambie o haya un `break`.
Una respuesta buena no se acepta («Sí» frente a «sí»).	Mayúsculas y minúsculas son distintas para Python.	Compara con `.lower()`: `if respuesta.lower() == "s":`

Los mensajes son los que da Python en online-python.com. En el editor de la prueba (`/pyodide`) son iguales, pero donde iría el nombre del archivo pone `<exec>` y a veces no salen las flechitas `^`: mira siempre el número de línea y la última línea, que dice el tipo de error.